

Document Intelligence Asynchronous Pipeline

Upload any document, get a job id back in milliseconds, and receive its text when processing finishes. Uses the claim-check pattern, survives crashes and runs fully on-device.

Team: Jose Angel Pech Xool · Valeria Nicol Hernandez León · Lorena Danae Perez Lopez ·

Julio Cesar de Aquino Castellanos · Gustavo Fuentes Marin · Ricardo Horta Sanchez

Professor: Dexter Gomez · **Course:** Trends in Data Science · **Date:** September 2026

Repository: github.com/joseangel0/document-intelligence-async-pipeline

Deliverable: source code (runs with `docker compose up --build`) and this report. The benchmark, chaos test results and every number in this report can be reproduced with the scripts in the repository.

- | | |
|--|---|
| 1. Product overview & why it matters for GenAI | 6. Part 4 · Queue design |
| 2. Architecture | 7. Part 5 · Failure handling (chaos & load tests) |
| 3. Part 1 · Ingestion & storage | 8. Part 6 · Monitoring & UI |
| 4. Part 2 · Job tracking | 9. Verification: requirement → evidence |
| 5. Part 3 · Document intelligence (two benchmarks) | 10. Trade-offs, limits & future work |
| | 11. Appendix · How to run & reproduce |

1. Product overview

Problem. Turning documents into machine-readable text is slow and unpredictable. A text file takes milliseconds, while a 30-page scan needs about a minute of OCR. Files also arrive in bursts, and many are malformed. A synchronous endpoint would hold the connection open, time out and lose work when a process dies.

Product. *DocIntel* is a self-hosted service with an HTTP API, a web UI and a monitoring dashboard. Clients push a document and get a job id back right away. They can then poll the job (or register a webhook) to follow its progress, and download the extracted content as plain text, **LLM-ready Markdown** and **retrieval chunks**, together with per-page details and metadata.

Why document intelligence is the foundation of GenAI

Most enterprise knowledge sits in files: scanned invoices, contracts in PDF, Word reports, spreadsheets, slide decks and e-mails with attachments. An LLM can only reason over that knowledge after it has been turned into text, and *how* it is turned into text sets the ceiling for every RAG system, agent or copilot built on top:

- **Structure carries meaning.** A price table flattened into loose lines ("Storage / GB-month / 5.00 / Support / hour ...") is ambiguous to a model. Markdown headings, lists and pipe tables keep the relationships.
- **Retrieval needs chunks with provenance.** Answers must cite *page 3, section "Payment terms"*. Chunks must respect section and table boundaries and fit a token budget.
- **Scale needs asynchrony and reliability.** Indexing a corpus means millions of pages. The pipeline must absorb bursts, never lose a document and say clearly what went wrong with the ones it could not read.
- **Privacy needs on-device processing.** Every model used here (OCR, layout, table structure, tokenizer, language ID) runs locally.

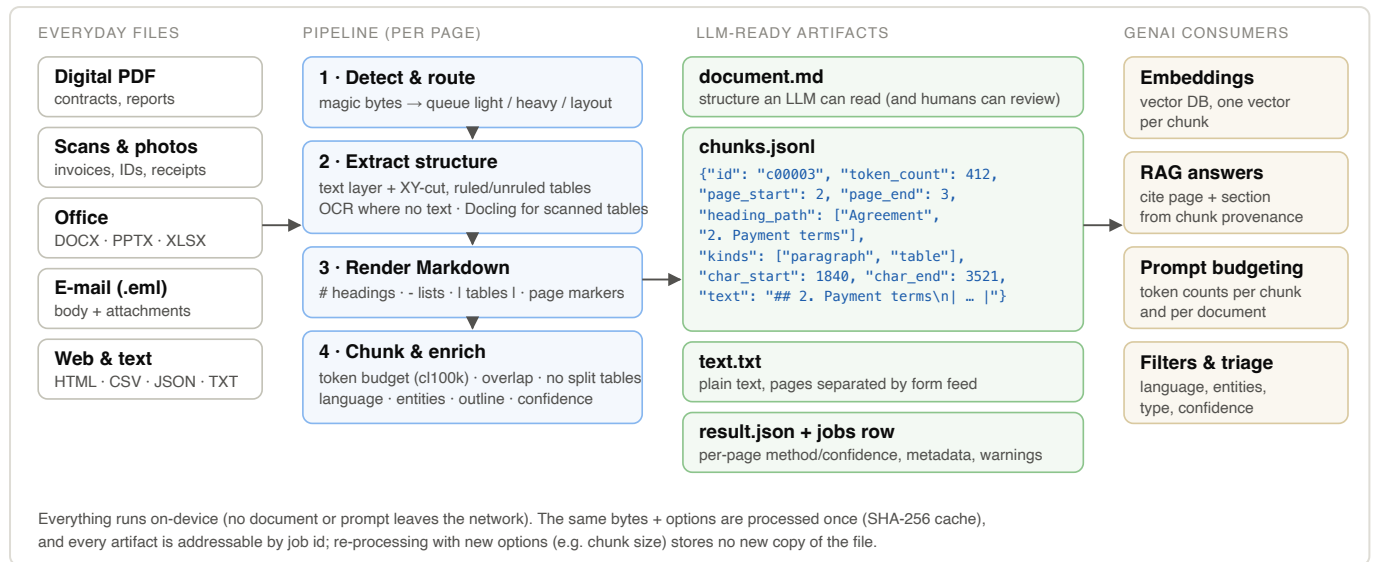


Figure 1. From everyday files to LLM-ready artifacts. Each job produces `document.md`, `chunks.jsonl`, `text.txt` and a structured `result.json`.

For API consumers

- POST `/v1/documents` → 202 + job id
- status, stage, progress %, queue position, warnings
- text, **Markdown**, **chunks (JSON/JSONL)**, per-page JSON, entities, webhook

Formats (by content, not extension)

- PDF (digital, scanned or mixed)
- PNG, JPG, TIFF, WebP, BMP, GIF
- TXT/MD/CSV/JSON/XML (any encoding), HTML, DOCX, PPTX, XLSX, RTF
- E-mail (.eml) with its attachments

Operational guarantees

- no job lost on worker, broker or full-stack crash
- every job ends SUCCEEDED / FAILED / CANCELLED
- 100% on-device, open-source components

Value-adding payload parameters. These were chosen because each one changes cost or behaviour in a way only the client knows about:

Parameter	Default	Why it exists
language	eng+spa	OCR accuracy depends on the language model. The default suits a bilingual (Mexican) user base. The value is validated against the installed traineddata, so a typo returns 422 instead of a failed job later.
ocr_mode	auto	auto runs OCR only on pages without a usable text layer. force re-OCRs PDFs whose text layer is garbage (a common scanner artefact). off saves minutes when only the embedded text matters.
ocr_engine	rapidocr	rapidocr is the most accurate on noisy scans and photos. tesseract is about 3× faster and uses about 4× less memory, and suits clean bulk scans (benchmark in §5).
pipeline	standard	layout sends PDFs and images to an optional Docling worker that rebuilds tables in scans and photos (table F1 1.00 vs 0.00), at ~8 s/page. It is a different cost/quality point that only the client can choose (§5.3).
chunk_size_tokens, chunk_overlap_tokens	512, 64	The right chunk size depends on the client's embedding model and context window. Both are validated (64–4096; overlap < size) and part of the cache key.
force	false	Uploads are content-addressed (SHA-256), so identical bytes with identical options return the stored result instantly (HTTP 200, cached_from_job_id). force=true bypasses this.

<code>extract_entities</code>	<code>true</code>	Emails, URLs, dates, amounts and phone numbers are cheap regex metadata that help triage invoices or forms.
<code>client_reference</code>	–	Correlates jobs with the caller's own records (filterable).
<code>callback_url</code>	–	Push instead of poll: a webhook fires on terminal state, retried with exponential backoff.

2. Architecture

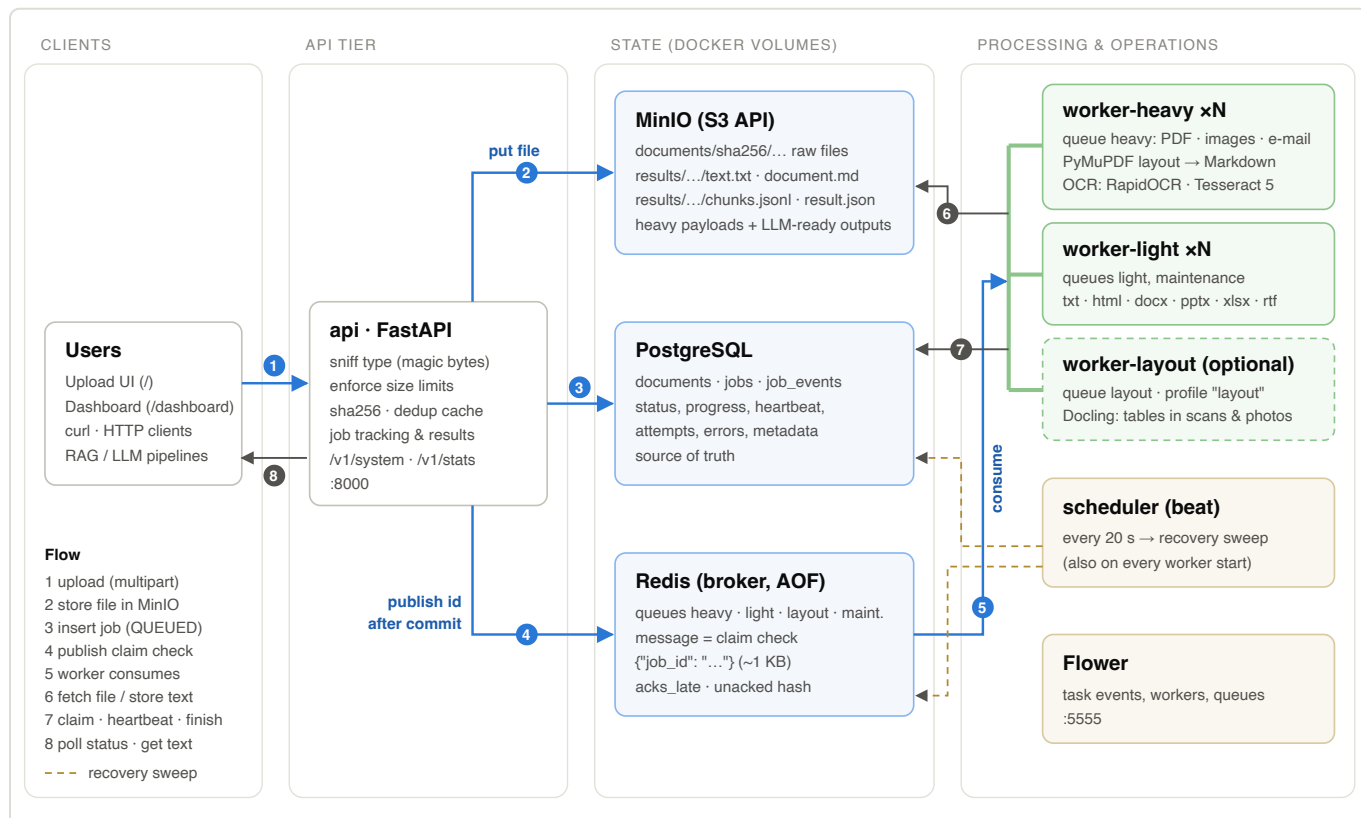


Figure 2. Components and the claim-check flow. Heavy bytes move only between the API, MinIO and the workers. The broker carries only a job id, and PostgreSQL is the single source of truth for job state. Everything runs as containers from one compose file, and the three stateful services keep their data in named volumes.

Service	Technology	Responsibility	Why this choice
api	FastAPI + Uvicorn	validation, claim-check in/out, tracking, UI, stats	OpenAPI docs for free, typed validation, streaming responses
worker-heavy	Celery (prefork)	PDFs, images, e-mails and Office files with pictures (may need OCR)	isolated so OCR can never starve cheap jobs; memory-capped
worker-light	Celery (prefork)	txt/html/office without pictures + maintenance tasks	sub-second jobs keep low latency during OCR bursts
worker-layout optional	Celery + Docling	PDFs/images with pipeline=layout	3 GB models kept out of the default image; started with <code>--profile layout</code>
scheduler	Celery beat	recovery sweep every 20 s	periodic safety net that does not depend on any single worker
redis	Redis 7, AOF	message broker only	recommended stack; AOF keeps queued messages across restarts

postgres	PostgreSQL 16	documents, jobs, job_events	transactions, row locks (SKIP LOCKED), JSONB metadata, aggregates for the dashboard
minio	MinIO (S3 API)	raw documents and extraction results	claim-check store; same code works on AWS S3/GCS/R2
flower	Flower 2	live Celery task and worker view	optional Part 6 requirement

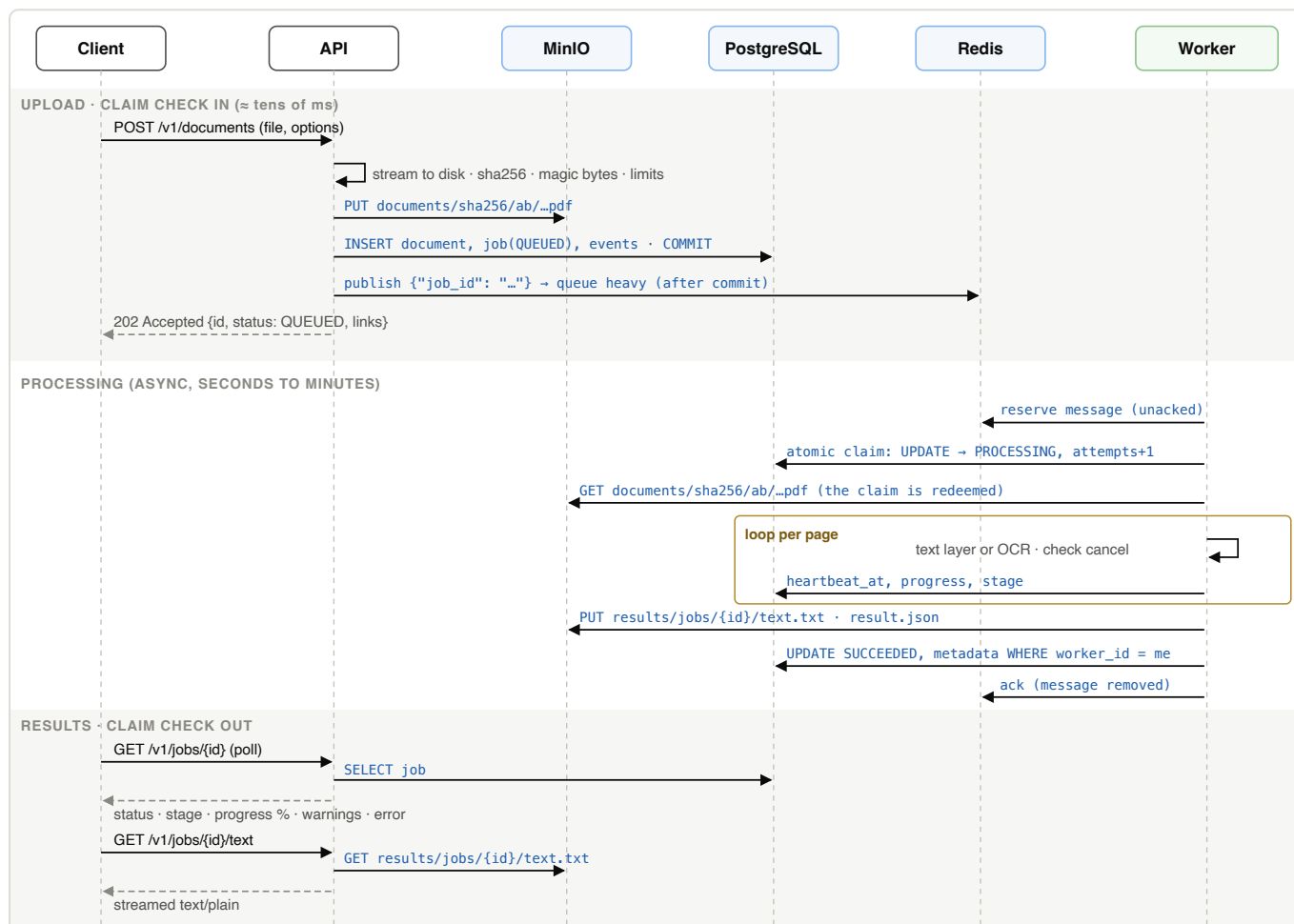


Figure 3. The claim-check lifecycle. The upload call returns as soon as the file is durable in object storage and the job row is committed. The message is published *after* the commit. Publishing first caused a real race during testing: a fast worker received the message before the row existed and dropped it.

3. Part 1 · Ingestion & storage

Claim-check implementation

- Stream and fingerprint.** The multipart body is streamed to a temp file in 1 MB chunks while a SHA-256 is computed. An outer ASGI middleware rejects oversized bodies *before* they are buffered, using `Content-Length` or by counting streamed bytes.
- Sniff the real type.** libmagic reads the magic bytes. ZIP containers are opened to tell DOCX, PPTX and XLSX apart, and text is checked with `charset-normalizer`. **Content wins over the extension:** a PNG named `.txt` is processed as an image, with a warning. Executables, archives and binary junk get **415** with the list of accepted extensions.
- Store the payload.** The object key is `documents/sha256/ab/<sha256>.ext`. Content addressing makes re-uploads free (HEAD first) and gives deduplication and caching at no extra cost.
- Record the claim check.** `documents` and `jobs` rows plus a `received` event are written in one transaction.

5. **Route and publish the ticket.** Only {"job_id": ...} is published. The queue depends on the kind and the options: `heavy` for PDFs, images and e-mails; `light` for text, HTML and Office files; Office files that embed more than 50 KB of pictures go to `heavy` because their images will be OCR'd; `pipeline=layout` goes to `layout`. The response is `202 Accepted` with a `Location` header and links.

Messages stay around 1 KB whatever the document size. That keeps Redis memory flat and makes redelivery cheap. It also means the payload is stored once, in a system built for large objects.

Upload limits and their rationale

Limit	Value	Reasoning
Request size	50 MB	Covers roughly 300 pages of 150 dpi scans or any realistic office file, while bounding API disk and bandwidth. Checked before buffering, so it cannot fill the disk.
PDF pages	500	At $\approx 0.3\text{--}2$ s of OCR per page, 500 scanned pages already approach the 15-minute time limit.
Image pixels	80 MP	Guards against decompression bombs, since a 76 KB PNG can declare 400 MP (tested). PDF pages are rendered with a DPI capped to the same budget.
Office uncompressed size	300 MB	DOCX/PPTX/XLSX are ZIP files, so this is a zip-bomb guard. XLSX is also capped at 2 M cells.
Processing time	900 s soft / 960 s hard	The soft limit fails the job with <code>TIMEOUT</code> and an actionable message. The hard limit kills a stuck native call.
Attempts	3	Enough to ride out a crash or outage. A "poison" document that crashes workers stops after 3 tries.
Queue TTL	24 h	Guarantees a terminal state even when no worker ever comes back (<code>EXPIRED</code>).

4. Part 2 · Job tracking

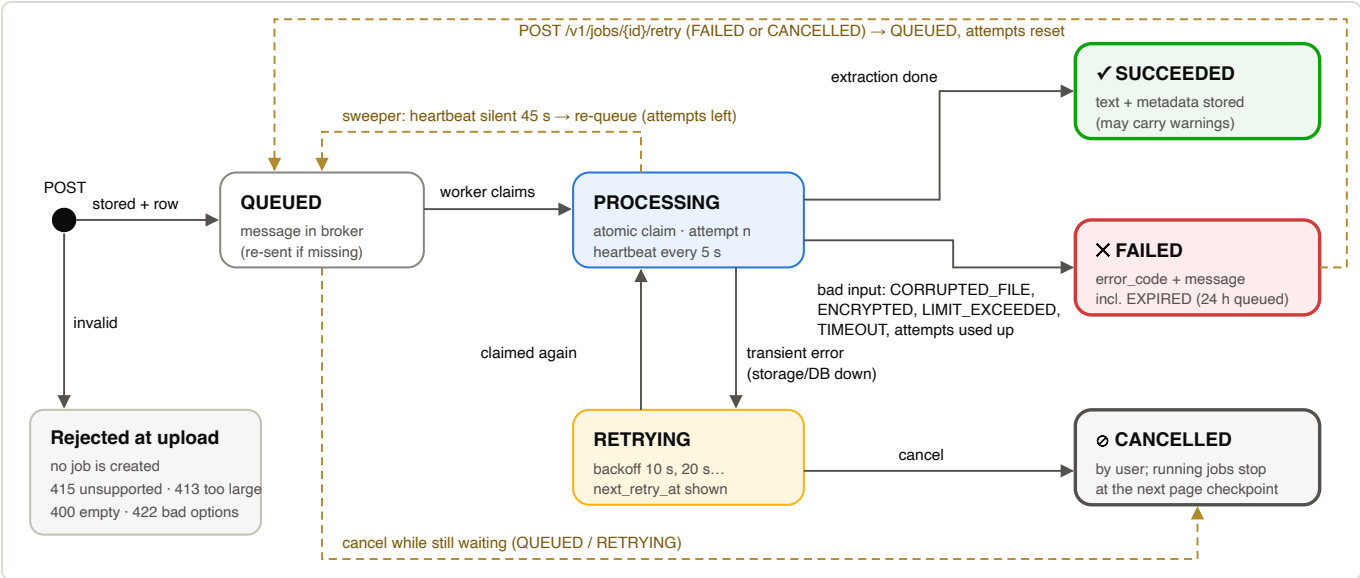


Figure 4. Job lifecycle. Solid arrows are worker transitions. Dashed arrows come from the recovery sweeper, the user or manual retries. Every path ends in one of three terminal states.

Data model (PostgreSQL)

Table	Key columns
-------	-------------

Tracking endpoints

Endpoint	Purpose
----------	---------

documents	sha256, original_filename, kind, mime_type, size_bytes, storage_bucket/key	GET /v1/jobs/{id}	status + plain-language message, stage ("ocr: page 7/30"), progress, attempts, queue position, warnings, error, timings
jobs	status, stage, stage_detail, progress, queue, options (JSONB), attempts/max_attempts, worker_id, heartbeat_at, next_retry_at, error_code/message, warnings (JSONB), result keys, text_preview, counts, metadata (JSONB), timestamps, cached_from_job_id, cancel_requested	GET /v1/jobs/{id}/events	timeline (received → dispatched → started → worker_lost → ...)
job_events	append-only timeline: level, event, human message, data (JSONB)	GET /v1/jobs	list/filter by status, kind, filename, client_reference
Document and job are separate because one file can be processed many times with different options, and the bytes are stored only once.		GET /v1/jobs/{id}/result	text + pages + metadata (409 with the reason if not ready)
		GET /v1/jobs/{id}/text	streamed plain text (download option)
		GET /v1/jobs/{id}/markdown	LLM-ready Markdown with page markers
		GET /v1/jobs/{id}/chunks	retrieval chunks with provenance (JSON, or JSONL download)
		POST .../cancel, .../retry	user control of the lifecycle

Why the database and not the Celery result backend? Celery's backend only knows `PENDING`. Its states are lost if Redis is flushed, and it cannot answer "how many OCR jobs failed today?". The job row is updated with a throttled progress write (at most 1 per second), a heartbeat every 5 s and ownership fencing (`WHERE worker_id = me`). Because of fencing, a worker the sweeper has already declared dead can never overwrite the newer attempt's result. `task_ignore_result=True` keeps Redis a pure transport.

5. Part 3 · Document intelligence

The goal is not only "text": it is **faithful, structured, traceable text** that an LLM can use. The pipeline is **hybrid and decided per page**: native parsers wherever the format carries text, layout analysis to recover structure, OCR only where there is no text, and an opt-in layout-model pipeline where tables must be rebuilt from pixels. Two benchmarks back the choices: **A** measures *what* text is recovered (OCR accuracy, text-layer fidelity) and **B** measures *how much structure* survives (headings, lists, tables).

5.1 Extraction strategy per format

Kind	Text and Markdown	Safeguards and extras
PDF (digital)	PyMuPDF text layer (<code>sort=True</code>) for <code>text</code> . For Markdown, a layout pass over the same page: headings from font size relative to the body size of the document, bullet/numbered lists (glyphs merged with their text by baseline), ruled tables via <code>find_tables()</code> , unruled tables from runs of column-aligned rows, and XY-cut reading order so two-column pages are read column by column. Wrapped headings are merged.	Word-coverage guard: if the Markdown keeps under 90% of the page's words, it falls back to plain text (seen on real forms). Damaged files are repaired with a warning; encrypted files and the page limit are reported.

PDF (scanned or mixed)	Pages with under 25 non-blank characters (or mostly images) are rendered at 300 dpi and OCR'd. RapidOCR boxes are deskewed and ordered with XY-cut; lines clearly taller than the body text become headings. With <code>pipeline=layout</code> , Docling rebuilds headings and tables.	In <code>auto</code> mode the richer of native/OCR text wins per page; the DPI is capped by the pixel budget; OCR confidence is reported per page.
Images	EXIF orientation fix, upscale small images, OCR (as above). Multi-page TIFF frames become pages.	Truncation check, 80 MP decompression-bomb limit; Tesseract OSD fixes 90/180/270° rotation when boxes are vertical or confidence is low.
DOCX	Body walked in document order: Title/Heading styles → <code>#</code> , list styles and numbering → list items, tables → pipe tables (merged cells de-duplicated). Pictures are OCR'd in place (scans pasted into Word).	Zip integrity and zip-bomb guard; icons/logos under 150 px skipped; <code>ocr_mode=off</code> respected with a warning.
PPTX / XLSX	One page per slide: <code>## Slide N: title</code> , bullets by paragraph level, tables, OCR of pictures, speaker notes. One page per sheet: a pipe table with cached formula values.	XLSX capped at 2 M cells; Markdown tables capped at 2 000 rows (plain text keeps everything) with a warning.
HTML · CSV · JSON · XML · TXT	HTML → Markdown (scripts/styles removed, tables kept); CSV/TSV → pipe table; JSON pretty-printed and XML in fenced code blocks; plain text escaped so a leading <code>#</code> never becomes a fake heading.	Encoding detection with confidence (Latin-1, UTF-16...); binary content disguised as text is rejected.
E-mail (.eml)	Headers as a table, HTML/plain body as Markdown, and every supported attachment extracted in the same job (PDF, image, Office...) as extra pages under <code>## Attachment: name</code> .	Unsupported attachments (e.g. <code>.exe</code>) are listed with the reason; attachment failures become warnings, not job failures.

5.2 Benchmark A · OCR accuracy and PDF text layers

Method (`benchmark/`, reproducible with one `docker run`). There are 53 samples with known ground truth, in English and Spanish (accents, ñ, ¿, emails, amounts, dates). Twelve image conditions: clean, blur, heavy noise, 1–4° skew, 120 dpi, JPEG q15, perspective "photo", white-on-colour, two columns, 6 pt font, invoice and 2-page scanned PDFs. There are seven kinds of born-digital PDF: single column, two columns, table, 20 pages, out-of-order content stream, glyph-by-glyph positioning and `/Rotate 90`. The **assignment PDF** itself was added as a real document, rasterised at 150 and 300 dpi. The metrics are CER and WER (Levenshtein on normalised text), a reading-order-insensitive bag-of-words F1, warm s/page (mean and p95), cold start, peak RSS and install size. Each engine ran in its own process with the same 4 threads on arm64 (10 CPU, 7.8 GB).

OCR engine	Macro CER	Macro WER	BoW F1	s/page (p95)	Cold start	Peak RSS	Install	Licence
Tesseract 5.3 (LSTM)	0.214	0.230	0.826	0.28 (0.56)	0.08 s	252 MB	15 MB	Apache-2.0
Tesseract + OpenCV preprocessing	0.080	0.126	0.912	0.33 (0.73)	0.10 s	348 MB	88 MB	Apache-2.0
RapidOCR, PP-OCRv6 small (ONNX)	0.001	0.005	0.997	0.91 (1.64)	0.34 s	1.1 GB	221 MB	Apache-2.0
RapidOCR, PP-OCRv5 latin	0.002	0.006	0.997	0.94 (1.69)	0.34 s	1.1 GB	204 MB	Apache-2.0
EasyOCR 1.7 (PyTorch)	0.043	0.138	0.966	6.96 (13.0)	2.73 s	6.4 GB	812 MB	Apache-2.0

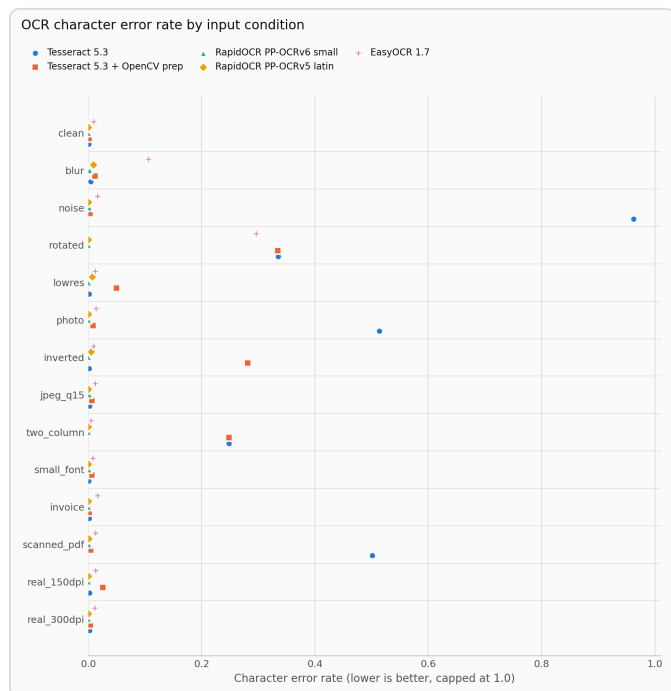


Figure 5. CER per condition. Raw Tesseract is excellent on clean input (0.001–0.002, including the real document) but collapses on noise (0.963, empty output), photos (0.514), noisy scanned pages (0.501), skew (0.335) and columns (0.248). OpenCV preprocessing fixes noise and photos but breaks white-on-colour text (0.281). RapidOCR stays at or below 0.009 in every condition.

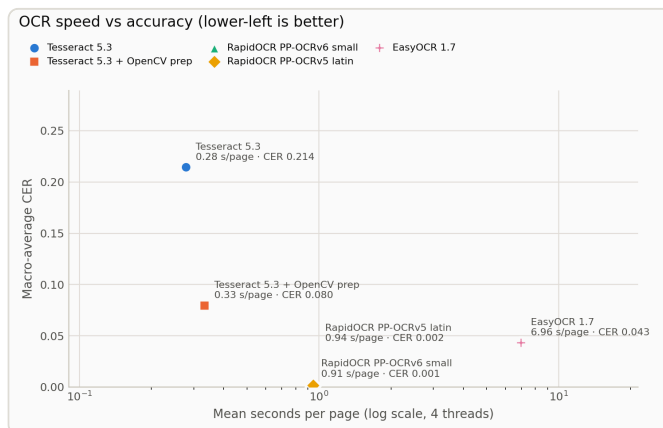


Figure 6. Speed vs accuracy (log x). EasyOCR is dominated: slower and less accurate than RapidOCR, and it needs 6.4 GB.

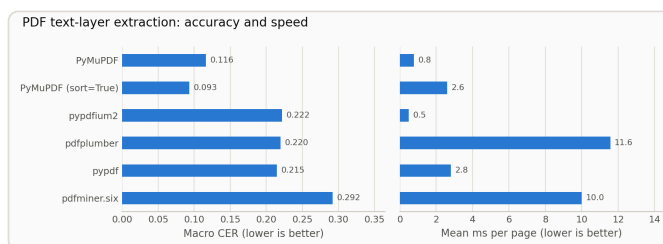


Figure 7. PDF text-layer extractors: macro CER and ms/page.

PDF extractor	Macro CER	ms/page	rotated page	glyph-positioned	out-of-order stream	table	Licence
PyMuPDF, sort=True	0.093	2.6	0.000	0.000	0.653	0.000	AGPL-3.0 / commercial
PyMuPDF (stream order)	0.116	0.8	0.000	0.000	0.813	0.000	AGPL-3.0 / commercial
pypdfium2	0.222	0.5	0.741	0.000	0.813	0.000	Apache-2.0 / BSD
pdfplumber	0.220	11.6	0.741	0.145	0.653	0.000	MIT
pypdf	0.215	2.8	0.000	0.690	0.813	0.000	BSD-3
pdfminer.six	0.292	10.0	1.249	0.000	0.147	0.648	MIT

All extractors were perfect on single-column, two-column and 20-page PDFs, and all returned **0 characters** for scanned pages. That validates the "few characters on this page, so OCR it" rule. No extractor recovers a content stream written out of visual order. That limitation is shared and is also why `ocr_mode=force` exists.

5.3 Benchmark B • Structure preserved for LLMs

Method (benchmark/run_structure_benchmark.py). Eleven inputs with ground-truth Markdown, in English and Spanish: three digital PDFs (report with H1–H3, ruled, borderless and 60-row multi-page tables; a Spanish report with a real two-column page; an invoice), two scanned PDFs, one photographed table and two DOCX. Three real public-domain IRS PDFs (Form W-9, Publication 1, Publication 15-T excerpt) are scored by word recall against the publisher's text layer, to catch silently dropped text. The metrics are heading F1 (text match, level ± 1), list-item recall, **table cell F1**, CER after stripping Markdown, and s/page. Candidates include the libraries themselves *and this service end to end through its API*, with both pipelines.

Candidate	Heading F1	List recall	Table F1	CER	s/page	Real docs BoW F1 (min)	Peak RSS	Licence
PyMuPDF get_text (baseline)	0.00	0.42	0.00	0.700	0.002	1.00 (1.00)	55 MB	AGPL / commercial
pymupdf4llm 0.0.24 (+ coverage guard)	0.41	0.44	0.32	0.668	0.025	0.98 (0.95)	82 MB	AGPL / commercial
pymupdf4llm 1.28 (+ layout model)	0.82	1.00	0.34	0.453	0.446	0.98 (0.96)	869 MB	AGPL / commercial
MarkItDown 0.1.7	0.33	0.61	0.46	0.525	0.038	0.98 (0.97)	218 MB	MIT
Docling 2.128 + RapidOCR	1.00	1.00	1.00	0.009	4.41	0.98 (0.95)	3.3 GB	MIT (models Apache/CDLA)
DocIntel service, standard pipeline	0.87	0.90	0.50	0.238	0.80	0.98 (0.96)	–	this project
DocIntel service, layout pipeline	1.00	1.00	1.00	0.009	7.74	0.98 (0.95)	–	this project

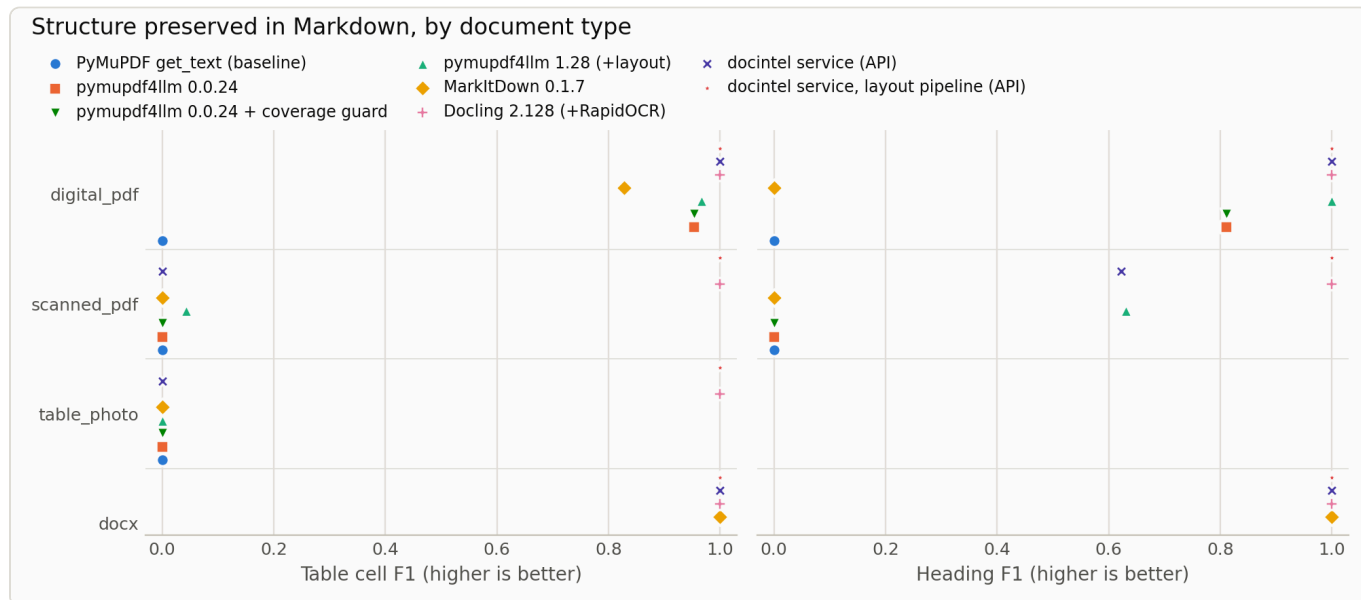


Figure 8. Table cell F1 and heading F1 per document type. Macro scores include scans, where tools without OCR score 0.

Heading F1 / table F1	digital PDF	scanned PDF	photo	DOCX
pymupdf4llm 0.0.24	0.81 / 0.95	0 / 0	– / 0	n/a
MarkItDown	0.00 / 0.83	0 / 0	– / 0	1.00 / 1.00
DocIntel standard	1.00 / 1.00	0.62 / 0.00	– / 0.00	1.00 / 1.00
DocIntel layout	1.00 / 1.00	1.00 / 1.00	– / 1.00	1.00 / 1.00

The benchmark changed the code. Its first run of the service found three defects in the standard pipeline: bullet glyphs on their own line (digital list recall 0.00), the two-column page read across columns (CER 0.285) and borderless tables emitted as loose lines. The fixes (baseline merging, XY-cut ordering, unruled-table detection, wrapped-heading merge, OCR heading detection) raised digital-PDF list recall to 1.00, table F1 from 0.96 to 1.00, two-column CER to 0.000 and scanned heading F1 from 0.00 to 0.62. Real documents kept a word recall of 0.96 or better throughout.

5.4 Decisions

PDF text: PyMuPDF (sort=True) + own layout pass

PyMuPDF was the only text-layer extractor without errors on rotated, glyph-positioned and table PDFs (benchmark A). For Markdown, the service's layout pass matched Docling on digital PDFs (1.00 / 1.00) at a fraction of the cost, beat pymupdf4llm 0.0.24 (0.81 / 0.95), and avoided pymupdf4llm 1.28, which is 18× slower, adds 800 MB of RAM and read the two-column page across columns. **Caveat:** AGPL; pypdfium2 is the permissive fallback.

OCR: RapidOCR default, Tesseract as fast option

RapidOCR (PP-OCRv6): CER 0.001 against 0.214 for raw Tesseract, at ≈1 s/page, fully offline. Its line classifier is disabled (it flipped upright lines); rotation uses Tesseract OSD. Tesseract is ≈3× faster with ¼ of the memory and as accurate on clean scans. EasyOCR was rejected: 7× slower, less accurate, 6.4 GB. In the service, a 30-page scan took 51.8 s with RapidOCR and 27.1 s with Tesseract.

Scanned tables: opt-in pipeline=layout (Docling)

No OCR-lines heuristic rebuilt tables from pixels (F1 0.00); only a table-structure model did (1.00). Docling costs 2.2 GB of install, ~3 GB RAM and 4–8 s/page, so it is **a separate worker, queue and image** (compose profile layout). The default path stays fast, and clients who need tables from scans (invoices, statements) opt in per document.

Office, HTML, e-mail: native parsers

DOCX structure from styles scored 1.00 on headings, lists and tables, the same as Docling and MarkItDown, with no models. HTML goes through markdownify, e-mail through the standard library. Office files with pictures are routed to the OCR pool so pasted scans are read.

5.5 LLM-ready outputs and chunking strategy

Decision	Choice and rationale
Canonical format	Markdown: models read it natively, it is compact (pipe tables cost far fewer tokens than HTML), humans can review it, and it keeps headings, lists and tables. Multi-page documents carry <code><!-- page: N --></code> markers, which are invisible when rendered but give page provenance.
Chunk boundaries	Structure first: a heading closes the current chunk once it holds ≥ ¼ of the budget (no micro-chunks). Paragraphs and lists are packed up to the budget; an oversized paragraph is split by sentences, then by tokens.
Tables	Never split mid-row. A table larger than the budget is split by rows, and the header row is repeated in every part , so each chunk is a self-describing table.
Overlap	Default 64 tokens, only between prose chunks of the same top-level section, cut at a word boundary. Tables and section changes get none, because overlap there would duplicate headers or mix topics.
Size & tokenizer	Default 512 tokens with tiktoken cl100k_base (baked into the image; falls back to chars/4 offline), per-request 64–4096. Token counts per chunk and per document let clients budget prompts and embedding costs.
Provenance	Each chunk: <code>id</code> , <code>index</code> , <code>page_start/page_end</code> , <code>heading_path</code> , <code>char_start/char_end</code> into <code>document.md</code> , <code>kinds</code> (paragraph, list, table, code), <code>token_count</code> , <code>overlap_tokens</code> . This is enough to cite, highlight and re-rank.

5.6 Where and how the extracted content is stored

Artifact	Location	Why there
Plain text (pages separated by form feed)	MinIO <code>results/jobs/{id}/text.txt</code>	Search indexing, diffing, legacy consumers; streamed.
Markdown	MinIO <code>.../document.md</code>	What LLMs and reviewers read; can be megabytes, so it belongs in object storage.
Retrieval chunks	MinIO <code>.../chunks.jsonl</code>	One JSON object per line, ready for bulk loading into a vector database.
Structured result: per-page text and Markdown, method (<code>text_layer</code> , <code>ocr:rapidocr</code> , <code>layout:docling</code>), confidence, timings, metadata, warnings, chunks	MinIO <code>.../result.json</code>	One self-contained, versionable document per job.
Queryable summary: preview, char/word/page/ token/chunk counts, language, metadata JSONB, warnings, result keys	PostgreSQL <code>jobs</code>	Powers listings, filters and the dashboard without touching object storage.

Keeping bytes in object storage and summaries in the database keeps PostgreSQL small and fast. The raw document stays content-addressed, so any job can be reprocessed with new options without re-uploading. A vector index is deliberately outside the service: `chunks.jsonl` is the hand-off to whichever embedding model and vector store the client uses.

5.7 Metadata worth extracting

Group	Fields	Use
Always	SHA-256, detected MIME and kind, size, pages, chars/words/lines, tokens, chunks, tables, outline (heading tree), extraction method per page, mean OCR confidence, timings	dedup, cost and context budgeting, quality triage, navigation
Language	ISO code + confidence (fastText lid.176, on-device)	route to language-specific models, pick the OCR language
Entities (regex)	emails, URLs, dates (ISO, numeric, Spanish and English month names), amounts (MXN/USD/EUR/\$), phone numbers	invoice and form triage without an LLM; metadata filters in RAG
PDF	title, author, subject, keywords, creator/producer, dates, encryption, repaired flag, page size, outline, text-layer vs OCR pages, tables detected, body font size	provenance, scanned-vs-digital routing
Image	format, dimensions, DPI, frames, EXIF (camera, date, software, orientation)	quality warnings, provenance
Office / HTML / e-mail	core properties, paragraph/heading/table/image counts, embedded images OCR'd, slide and sheet names; HTML title/lang/description; e-mail from/to/cc/date/subject and per-attachment status	search facets, display, audit

6. Part 4 · Queue design (Redis + Celery)

Setting	Value	Effect / reason
Queues	<code>heavy</code> , <code>light</code> , <code>layout</code> , <code>maintenance</code>	Routing is decided at upload from the detected kind and options (§3). A burst of 500-page scans cannot delay a 1 KB text file, and each pool can be scaled on its own (<code>--scale worker-heavy=3</code>).

<code>task_acks_late</code>	True	The message is acknowledged only after the task finishes, so a crash leaves it unacked and it gets redelivered.
<code>task_reject_on_worker_lost</code>	True	If a pool child is OOM-killed, the message is queued instead of being marked failed.
<code>worker_prefetch_multiplier</code>	1	A busy worker doesn't reserve messages it can't start, so another replica can take them.
<code>visibility_timeout</code>	hard limit + 300 s	Redis redelivers unacked messages only after the longest legitimate task has had time to finish.
Concurrency	heavy 2 processes × <code>OCR_THREADS=2</code> (4 GB limit), light 4	Threads per process are pinned (<code>OMP_THREAD_LIMIT</code> , <code>ONNX</code> intra-op), so processes × threads ≈ cores reserved for OCR with no oversubscription. RapidOCR peaks at ~1.1 GB per process.
<code>worker_max_tasks_per_child</code>	200	Recycles processes to contain leaks in native libraries (MuPDF, Leptonica).
Time limits	soft 900 s / hard 960 s	A runaway job becomes a clear <code>TIMEOUT</code> failure instead of a stuck slot.
Task id	<code><job_id>.</code> <code><dispatch_n></code>	Lets the sweeper map broker messages back to jobs, and keeps each re-dispatch distinguishable in Flower.
Redis persistence	AOF, <code>everysec</code>	Queued messages survive a broker restart, losing at most 1 s of messages. The sweeper covers even that.

At-least-once delivery plus an idempotent claim gives effectively-once processing. A worker starts a job only through an atomic `UPDATE jobs SET status='PROCESSING', attempts=attempts+1 ... WHERE id=? AND status IN ('QUEUED','RETRYING') AND attempts < max_attempts RETURNING id`. Duplicate deliveries, redelivered messages and re-dispatches find the row not claimable and are acknowledged in about 3 ms without doing any work.

7. Part 5 • Failure handling

Failure matrix

Failure	Mechanism	What the user sees
Unsupported, empty or oversized upload	Magic-byte sniffing, size guard before buffering	415 / 400 / 413 with <code>error.code</code> , message and accepted extensions. No job is created.
Corrupted / truncated / encrypted / bomb file	Typed extractor errors (<code>retryable=False</code>), so there is no pointless retry. MuPDF repairs damaged PDFs when it can.	<code>FAILED + CORRUPTED_FILE</code> , <code>ENCRYPTED_DOCUMENT</code> or <code>LIMIT_EXCEEDED</code> , or <code>SUCCEEDED</code> with a "PDF was repaired" warning.
Worker process or container killed mid-job	Heartbeat stops. The sweeper finds <code>PROCESSING</code> with a heartbeat older than 45 s, re-queues and re-dispatches. The message is also unacked (<code>acks_late</code>).	Timeline shows <code>worker_lost</code> and attempt 2/3. A heartbeat-age warning appears while waiting.
Poison document (crashes every time)	Attempts are counted in the claim; after 3 the sweeper fails the job.	<code>FAILED WORKER_LOST</code> : "stopped responding on 3 of 3 attempts... makes the worker crash or run out of memory".
Whole stack killed	Durable volumes. On start, every worker runs a sweep (advisory-locked) and beat continues every 20 s.	Jobs resume by themselves: running ones on attempt 2, queued ones from the AOF-restored queue.

Broker loses messages / publish fails	The sweeper compares QUEUED jobs against the Redis lists and <code>unacked</code> hash and re-dispatches missing ones. Duplicates are harmless.	Upload still returns 202 with a "queue temporarily unavailable" warning. Events show <code>dispatch_failed</code> / <code>message_missing</code> .
Object storage / DB outage	API maps errors to 503 + <code>Retry-After</code> . Workers classify them as transient: <code>RETRYING</code> with exponential backoff (10 s, 20 s...).	<code>STORAGE_UNAVAILABLE</code> / <code>DATABASE_UNAVAILABLE</code> . The job shows <code>next_retry_at</code> and the attempt counter.
Slow or stuck processing	Soft/hard time limits; cancellation checked per page. The soft limit is also recognised when a native library wraps it (ONNX Runtime raised it as <code>ONNXRuntimeError</code> , which the chaos test caught being retried 3×), by cause chain and elapsed time.	<code>FAILED TIMEOUT</code> on attempt 1 with advice (split, <code>ocr_engine=tesseract</code> , <code>ocr_mode=off</code>), or <code>CANCELLED</code> .
Cancel requested, then the worker dies	The sweeper honours <code>cancel_requested</code> on stale jobs instead of re-queueing them. Found in the audit: such a job would otherwise sit QUEUED unclaimable until the 24 h expiry.	<code>CANCELLED</code> within ~1 min, with the reason in the timeline.
Stored file deleted (retention, manual)	<code>NoSuchKey</code> is a non-retryable <code>ObjectMissingError</code> , not a transient storage error.	<code>FAILED DOCUMENT_MISSING</code> on attempt 1; result downloads return 410.
No workers at all	Queue TTL expiry; readiness endpoint	<code>/v1/system</code> : "No workers online: jobs will wait...". After 24 h: <code>FAILED EXPIRED</code> .
Low-quality result	Per-page confidence, text-layer detection, encoding confidence	Warnings: low OCR confidence on pages 3–5, mixed document, no text found, extension mismatch, rotation corrected...

Chaos test results

`scripts/chaos_test.py` injects each failure into the running compose stack and asserts the outcome through the public API only.

Scenario	Result	Expected	Observed (from the public API)	Time
<code>rejected_inputs</code>	✓ PASS	unsupported/empty → 4xx at upload; damaged files → terminal state with explicit error code (repairable PDFs succeed with a warning)	<code>program.exe</code> → HTTP 415; <code>archive.zip</code> → HTTP 415; <code>empty.txt</code> → HTTP 400; <code>binary_named_as.txt</code> → HTTP 415; <code>corrupted.pdf</code> → SUCCEEDED (warnings: 3); <code>garbage.pdf</code> → FAILED <code>CORRUPTED_FILE</code> ; <code>truncated.jpg</code> → FAILED <code>CORRUPTED_FILE</code> ; <code>encrypted.pdf</code> → FAILED <code>ENCRYPTED_DOCUMENT</code> ; <code>huge_400mp.png</code> → FAILED <code>LIMIT_EXCEEDED</code>	5s
<code>worker_kill</code>	✓ PASS	job re-queued after heartbeat timeout and SUCCEEDED on attempt 2	SUCCEEDED after 2 attempts; recovered+finished 109s after kill; events: received → dispatched → started → <code>worker_lost</code> → dispatched → started → succeeded	119s
<code>cancel_running</code>	✓ PASS	a running job stops at the next page and ends CANCELLED	CANCELLED at 11%	6s

broker_data_loss	✓ PASS	sweeper detects jobs missing from the broker and re-dispatches them	['SUCCEEDED', 'SUCCEEDED', 'SUCCEEDED'] in 61s; message_missing events: True	64s
broker_down_on_upload	✓ PASS	upload still accepted (202); job dispatched automatically when the broker returns	HTTP 202; final SUCCEEDED; events: received → dispatched → dispatch_failed → message_missing → dispatched → started → succeeded	18s
storage_down_during_processing	✓ PASS	job goes RETRYING with backoff, then SUCCEEDED; uploads during the outage get 503 STORAGE_UNAVAILABLE	upload during outage → HTTP 503 STORAGE_UNAVAILABLE; job SUCCEEDED after 2 attempts; events: received → dispatched → started → retry_scheduled → dispatched → started → succeeded	18s
database_down	✓ PASS	HTTP 503 DATABASE_UNAVAILABLE while down, /v1/system reports degraded, recovers automatically	/v1/jobs → 503 DATABASE_UNAVAILABLE; /v1/system → 503 ['Database unavailable: uploads and status queries will fail until it recovers.']; after restart → 200	11s
document_missing	✓ PASS	FAILED DOCUMENT_MISSING on the first attempt (retrying cannot help)	FAILED DOCUMENT_MISSING (attempts 1)	5s
cancel_while_worker_dead	✓ PASS	CANCELLED by the sweeper once the heartbeat is stale (no re-queue)	CANCELLED (attempts 1); events: received → dispatched → started → cancel_requested → cancelled	68s
queue_expiry	✓ PASS	QUEUED job older than the queue TTL → FAILED EXPIRED	sweep {'requeued': 0, 'failed': 0, 'redispatched': 0, 'expired': 1, 'broker_checked': True} → FAILED EXPIRED	35s
processing_timeout	✓ PASS	FAILED TIMEOUT with actionable advice; no retry loop	FAILED TIMEOUT after 5.28s, attempts 1: Processing exceeded the time limit (5s). Large scanned documents are slow to OCR: split the document	39s
poison_pill	✓ PASS	after 3 crashed attempts the job ends FAILED WORKER_LOST with an explanation	3 kills → FAILED WORKER_LOST after 3 attempts: The worker processing this document stopped responding (last heartbeat 56s ago) on 3 of 3 attempts. This usually means t	185s
stack_kill	✓ PASS	every job (queued, running, finished) reaches SUCCEEDED after a full restart	scanned_report_30p.pdf: PROCESSING→SUCCEEDED (attempts 2); scanned_letter.pdf: SUCCEEDED→SUCCEEDED (attempts 1); mixed_text_and_scan.pdf: PROCESSING→SUCCEEDED (attempts 2); report.docx: SUCCEEDED→SUCCEEDED (attempts 1); invoice_photo.jpg: QUEUED→SUCCEEDED (attempts 1)	107s

13/13 passed, run (2026-09-16 23:11) on the compose stack (arm64, OrbStack, layout profile running). Worker loss is detected after 45 s without a heartbeat and the sweeper runs every 20 s, so recovery includes up to ~65 s of detection plus the re-run. The `processing_timeout` scenario first **failed** during development: a soft limit wrapped by ONNX Runtime was retried 3× as an internal error. That bug was fixed. `document_missing` and `cancel_while_worker_dead` cover two bugs found in the audit.

Burst load test

`scripts/load_test.py` uploads a mixed burst of documents with 16 parallel clients, so documents arrive faster than they can be processed. It then checks that every job terminates, and that cheap documents are not starved by the OCR backlog.

- Documents uploaded: **122** by 16 parallel clients; accepted **122** in 1.0 s (upload latency p50 116 ms, p95 238 ms).
- Terminal states: {'SUCCEEDED': 122}; still pending after the deadline: **0**.
- All jobs drained in **50 s**: 175 pages, 28,133 LLM tokens, throughput 145 documents/min.

Queue	Jobs	Wait p50 (s)	Wait p95 (s)	Wait max (s)	Mean processing (s)	Peak depth
heavy	59	35.3	39.5	41.8	1.46	57
light	63	0.6	0.8	0.8	0.07	49

Reading. Uploads stay fast because the API only stores the file and a row. The OCR queue backs up, which is expected with 2 OCR processes on a laptop. Meanwhile the light queue keeps sub-second waits: routing by cost protects cheap documents. Throughput scales linearly with `--scale worker-heavy=N`.

8. Part 6 · Monitoring & UI

Monitoring works on three levels. Each one answers a different question.

Tool	Question it answers	Contents
Dashboard <code>/dashboard</code>	"Is the pipeline healthy and keeping up?" (product and operations view)	dependency health pills; tiles for received, success rate, in-flight, p50/p95 processing time, queue wait, pages, LLM tokens and chunks , jobs with warnings; throughput chart (succeeded vs failed per interval); queue depths and per-worker busy slots; jobs and average time by document type; failures by error code; reliability events (worker losses recovered, lost messages re-dispatched, automatic retries, uploads while the broker was down); active jobs with heartbeat age; recent failures. Window 15 min–7 days, auto-refresh 3 s.
Flower :5555	"What is Celery doing right now?" (task level)	per-task state, runtime and args (the job id), worker pools, broker queues, rate limits and remote control
GET <code>/v1/system</code>	"Can I use the service?" (for machines and the UI banner)	DB, object storage, broker and worker status with human messages; HTTP 503 when degraded

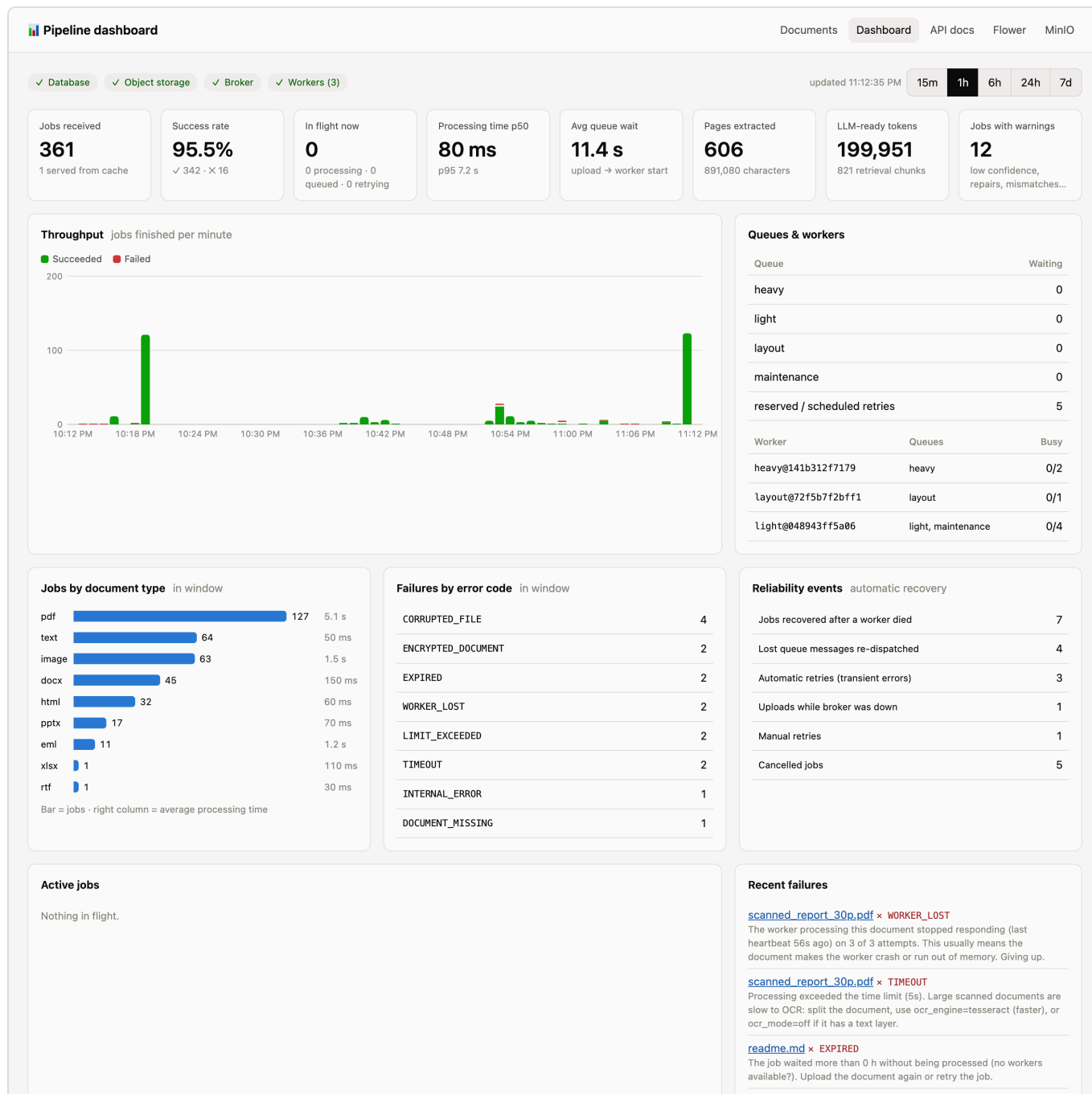


Figure 9. Monitoring dashboard after the chaos and load test runs. The reliability panel counts the automatic recoveries, and the failure panel shows the explicit error codes of the malformed inputs.

Document Intelligence
All systems operational
Documents
Dashboard
API docs
Flower

Upload documents
max 50 MB · 500 PDF pages

Drop files here or click to choose

pdf · png · jpg · jpeg · tif · tiff · webp · bmp · gif · txt · md · markdown · csv · tsv · log · json · xml · html · htm · xhtml · docx · pptx · xlsx · eml · rtf

OCR languages

eng+spa

OCR mode

auto (only pages with

OCR engine

rapidocr

Pipeline

standard (fast)

Chunk size (tokens)

512

Chunk overlap (tokens)

64

Client reference

optional

☒ Extract entities
☐ Re-process duplicates

Jobs

Filter by filename

All statuses

Document	Status	Progress	Pages	Time	Created
readme.md text · 685 B · 1	SUCCEEDED	done	1	50 ms	10:53:37 PM
contract_scanned.pdf pdf · 168.7 KB	SUCCEEDED	done	1	1.5 s	10:53:35 PM
contract_scanned.pdf pdf · 168.7 KB	SUCCEEDED	done	1	7.7 s	10:53:27 PM
invoice_digital.pdf pdf · 2.7 KB	SUCCEEDED	done	2	50 ms	10:53:24 PM
prices.csv text · 59 B	SUCCEEDED	done	1	40 ms	10:53:23 PM
readme.md text · 685 B	SUCCEEDED	done	1	30 ms	10:53:21 PM
scanned_report_30p.pdf pdf · 4.4 MB	CANCELLED	done	—	7.1 s	10:53:21 PM
dup.txt text · 47 B	SUCCEEDED	done	1	30 ms	10:53:14 PM
dup-again.txt text · 47 B	SUCCEEDED	cached	1	0 ms	10:53:14 PM
dup.txt text · 47 B	SUCCEEDED	done	1	30 ms	10:53:14 PM
blank_page.pdf pdf · 460 B · 2	SUCCEEDED	done	1	710 ms	10:53:13 PM
corrupted.pdf pdf · 938 B · 3	SUCCEEDED	done	2	780 ms	10:53:12 PM
text_named_as.png	SUCCEEDED	done	1	40 ms	10:53:12 PM

contract_with_scan.docx

SUCCEEDED

Text extracted successfully.

.md

chunks.jsonl

.txt

{ } JSON

Original

Job

6ee9906a-fea0-45b2-8b9a-324adfe18cbc

Type

docx · application/vnd.openxmlformats-officedocument.wordprocessingml.document · 100.0 KB

Options

language=eng+spa

ocr_mode=auto

pipeline=standard

ocr_engine=rapidocr

extract_entities=true

chunk_size_tokens=512

chunk_overlap_tokens=64

Queue

heavy · attempt 1/3

Timing

waited 10 ms · processed in 950 ms

Result

1 page(s) · 138 words · 982 chars · 368 tokens · 2 chunks · 1 tables · language en (73%)

▼ Entities

emails

billing@universidad.example.mx

urls

https://pay.example.com/inv/2026-0917

dates

2026-09-17 17/10/2026

amounts

\$3,600.00 \$250.00 \$1,150.00 \$5,000.00 \$800.00 \$5,800.00

phone numbers

+52 999 123 4567

► Document metadata

► Timeline (4)

Extracted content

Markdown (LLM)

Plain text

Chunks (2)

Service Agreement

1. Scope of services

The provider will digitise and index the client's archive of invoices and contracts. All processing runs on premises; no document leaves the client's network.

2. Payment terms

Invoices are payable within 30 days. Late payments accrue interest of 1.5% per month.

3. Deliverables

- Searchable text for every page

- Markdown with tables preserved

4. Price list

Service	Unit	Price (MXN)
OCR page	page	3.00
Storage	GB-month	5.00
Support	hour	1,150.00

Figure 10. Upload UI: options (OCR, pipeline, chunking), a live job list and job detail. The detail shows entities, metadata, the timeline and the extracted content in three views: **Markdown (LLM)**, plain text and chunks. The downloads are `.md`, `chunks.jsonl`, `.txt` and JSON.

 Flower Workers Tasks Broker Documentation 						
Show 15 tasks		Search:				
Name	UUID	State	args	kwargs	Result	
docintel.recover_jobs	52b7bed5-8680-4db9-bb47-14ac59955150	SUCCESS	()	{}	{ 'requeued': 0, 'failed': 0 }	
docintel.recover_jobs	ff433ce3-3070-48fd-9bb3-f04bd0473a9c	SUCCESS	()	{}	{ 'requeued': 0, 'failed': 0 }	
docintel.process_document	d07c7674-7946-4347-9da9-99ba5f8cb5db.1	SUCCESS	()	{ 'job_id': 'd07c7674-7946-4347-9da9-99ba5f8cb5db...' }	{ 'job_id': 'd07c7674-7946-4347-9da9-99ba5f8cb5db...' }	
docintel.process_document	ffe2576e-362f-4a62-b1ed-13599783b438.1	SUCCESS	()	{ 'job_id': 'ffe2576e-362f-4a62-b1ed-13599783b438' }	{ 'job_id': 'ffe2576e-362f-4a62-b1ed-13599783b438' }	
docintel.process_document	70c216ce-4e89-4bc2-a4ca-1ac98f5521c1.1	SUCCESS	()	{ 'job_id': '70c216ce-4e89-4bc2-a4ca-1ac98f5521c1' }	{ 'job_id': '70c216ce-4e89-4bc2-a4ca-1ac98f5521c1' }	
docintel.process_document	e986b23d-b249-4e8b-bc18-41b768037535.1	SUCCESS	()	{ 'job_id': 'e986b23d-b249-4e8b-bc18-41b768037535...' }	{ 'job_id': 'e986b23d-b249-4e8b-bc18-41b768037535...' }	
docintel.process_document	b2e37ded-cfc1-47ab-b56b-a7898a4b83da.1	SUCCESS	()	{ 'job_id': 'b2e37ded-cfc1-47ab-b56b-a7898a4b83da...' }	{ 'job_id': 'b2e37ded-cfc1-47ab-b56b-a7898a4b83da...' }	
docintel.process_document	aeb7ad79-7b1f-4e52-b7b0-0da2e07f5874.1	SUCCESS	()	{ 'job_id': 'aeb7ad79-7b1f-4e52-b7b0-0da2e07f5874' }	{ 'job_id': 'aeb7ad79-7b1f-4e52-b7b0-0da2e07f5874' }	
docintel.process_document	3ba4a016-bfda-4e56-a1eb-c05ff11a2468.1	SUCCESS	()	{ 'job_id': '3ba4a016-bfda-4e56-a1eb-c05ff11a2468' }	{ 'job_id': '3ba4a016-bfda-4e56-a1eb-c05ff11a2468' }	
docintel.process_document	a0c44931-1776-4e81-8f68-b2ededbc173c.1	SUCCESS	()	{ 'job_id': 'a0c44931-1776-4e81-8f68-b2ededbc173c' }	{ 'job_id': 'a0c44931-1776-4e81-8f68-b2ededbc173c' }	
docintel.process_document	cb9c5a45-88f6-41bf-9d15-e70ab21dd984.1	SUCCESS	()	{ 'job_id': 'cb9c5a45-88f6-41bf-9d15-e70ab21dd984' }	{ 'job_id': 'cb9c5a45-88f6-41bf-9d15-e70ab21dd984' }	
docintel.process_document	45991a1d-1b8f-4841-bb79-c334bbfdf238.1	SUCCESS	()	{ 'job_id': '45991a1d-1b8f-4841-bb79-c334bbfdf238' }	{ 'job_id': '45991a1d-1b8f-4841-bb79-c334bbfdf238' }	
docintel.process_document	f9256016-4c6f-472b-835b-f3e1ae029e2b.1	SUCCESS	()	{ 'job_id': 'f9256016-4c6f-472b-835b-f3e1ae029e2b' }	{ 'job_id': 'f9256016-4c6f-472b-835b-f3e1ae029e2b' }	
docintel.process_document	460d9916-8cd6-4edd-8ca4-2d986cc337cc.1	SUCCESS	()	{ 'job_id': '460d9916-8cd6-4edd-8ca4-2d986cc337cc...' }	{ 'job_id': '460d9916-8cd6-4edd-8ca4-2d986cc337cc...' }	
docintel.process_document	62f0d17d-0f0e-4268-9528-083137a2266e.1	SUCCESS	()	{ 'job_id': '62f0d17d-0f0e-4268-9528-083137a2266e...' }	{ 'job_id': '62f0d17d-0f0e-4268-9528-083137a2266e...' }	

Figure 11. Celery Flower. Task ids are `<job_id>.<dispatch_n>`, so re-dispatches after a recovery are visible as `.2`, `.3`.

9. Verification: requirement → evidence

Every requirement of the assignment is backed by an automated check that runs against real containers. The same suites run in **GitHub Actions on x86_64** from a clean checkout. That proves the single-command setup on a machine other than the development laptop (arm64).

Suite	What it exercises	Checks	Result
Unit (<code>pytest</code> , app image)	format detection and spoofing, every extractor (text and Markdown), OCR engines, rotation, e-mail attachments, embedded-image OCR, chunker invariants, typed errors	47	✓ all pass
Unit (layout image)	Docling pipeline: scanned table rebuilt, digital PDF, image, non-applicable formats	3 (+15 shared)	✓ all pass
Integration (<code>tests/integration</code>)	through HTTP: 17 formats end to end (text, Markdown, chunks, JSONL, result, events); 415/400/413/422 errors; failed-file messages; warnings; dedup cache; cancel and retry; webhook delivery; filters; downloads; system/formats/stats/UI; layout pipeline	40	✓ all pass
Chaos (<code>scripts/chaos_test.py</code>)	13 failure injections (table in §7)	13	✓ 13/13 pass
Load (<code>scripts/load_test.py</code>)	mixed burst; all terminal; upload latency; queue isolation	1	✓ pass (122/122 succeeded)
Benchmarks A and B	parser/OCR accuracy and structure, with the service measured end to end	–	reproducible with <code>docker run</code>

CI (GitHub Actions, x86_64)	build → unit → <code>docker compose up --wait</code> → integration → 9 chaos scenarios → load test	– ✓ green (run 35184845357, 12 min): unit 47, integration 39 + 1 skipped (layout profile not started in CI), chaos 9/9, load 59/59
-----------------------------	---	---

On the CI runner (2–4 shared vCPUs, no warm caches) the load test also passed. The light-queue wait was higher there (p95 9.4 s against 0.8 s on the laptop) because every container shares those few cores, but no job was lost or left pending.

Requirement (assignment)	Implementation	Evidence
Push documents; accept PDF, PNG/JPG, TXT; reject unsupported	POST /v1/documents, content sniffing (§3)	integration test_format_end_to_end ×17, test_rejected_uploads; chaos rejected_inputs
Claim-check pattern	MinIO payload + {job_id} message after commit (Fig. 3)	integration end-to-end tests; broker_data_loss (messages rebuilt from DB claims)
Additional formats and payload parameters	HTML, DOCX, PPTX, XLSX, RTF, CSV/JSON/XML, EML; 10 parameters (§1)	test_format_end_to_end, test_invalid_options, test_duplicate_upload_is_served_from_cache, test_webhook_is_delivered
Reasonable upload limits	50 MB (pre-buffer), pages, pixels, zip size, time (§3)	test_upload_limit_is_enforced_before_buffering; huge_400mp.png; processing_timeout
Job tracking endpoints; DB lifecycle	jobs, events, result, text, markdown, chunks, cancel, retry; PostgreSQL model (§4)	test_result_not_ready_then_cancel_then_retry, test_listing_filters_and_not_found
Best parsing methods, supported by a benchmark	hybrid per-page pipeline; RapidOCR; PyMuPDF + layout pass; opt-in Docling (§5)	Benchmarks A and B (tables in §5.2–5.3), including the service itself
How/where to store text; extra metadata	MinIO artifacts + PostgreSQL summary; metadata catalogue (§5.6–5.7)	integration checks of /text, /markdown, /chunks, /result
On-device, open source	all models baked into images; no external API calls	CI runs with no credentials; licences in §5
Redis + Celery	queues, acks_late, prefetch 1, beat (§6)	every chaos scenario
Unsupported / corrupted files	typed errors, no pointless retries	test_bad_files_fail_with_explicit_error; rejected_inputs
Workers or whole stack killed mid-processing; no job lost	heartbeats, sweeper, idempotent claim, AOF, volumes	worker_kill, stack_kill, broker_data_loss, broker_down_on_upload
No document non-terminal forever	max attempts, time limits, TTL expiry, cancel-on-stale	poison_pill, processing_timeout, queue_expiry, cancel_while_worker_dead, document_missing
Warnings, errors, status information	error codes, warnings, timeline, /v1/system, UI banners	test_warnings_are_reported; database_down, storage_down_during_processing
Flower, dashboard, simple UI (optional)	§8	test_system_formats_stats_and_ui; screenshots
Single docker compose command; report	<code>docker compose up --build</code> ; this PDF	CI job on a clean x86_64 runner

10. Trade-offs, limits & future work

Decision	Cost we accept	Mitigation / next step
Redis as broker (not RabbitMQ/SQS)	Weaker delivery guarantees: unacked messages come back only after the visibility timeout, and FLUSHALL loses everything.	Postgres is the source of truth and the sweeper re-dispatches from it (tested). Swap to RabbitMQ quorum queues if the message volume grows.
Heartbeat-based loss detection	Recovery latency is 45–65 s. A 60 s network partition could cause a redundant re-run.	Ownership fencing makes a late worker's writes no-ops. Both timeouts are env-tunable.
Job-level (not page-level) retries	A crash at page 29/30 re-OCRs the whole document.	Split large PDFs into page-range subtasks (Celery chord) with per-page checkpoints in object storage.
Polling API	Clients must poll.	Webhook (<code>callback_url</code>) exists. Server-Sent Events on <code>/v1/jobs/{id}/stream</code> would be the next step.
PyMuPDF (AGPL)	Licence obligations if the service is distributed closed-source.	pypdfium2 fallback (benchmarked) or a commercial MuPDF licence.
RapidOCR default	1.1 GB peak per process, $\approx 3\times$ slower than Tesseract, and it sometimes merges words on photos.	<code>ocr_engine=tesseract</code> for clean bulk scans; worker memory limit and concurrency sized for it.
Standard pipeline on scans	Tables in scans/photos are not rebuilt (table F1 0.00); scanned headings are only partly recovered (F1 0.62).	Opt-in <code>pipeline=layout</code> (F1 1.00). Next step: automatically escalate pages whose OCR geometry looks tabular.
Docling as a separate profile	+2.2 GB image, ~ 3 GB RAM per process, 4–8 s/page on CPU; not started by default.	Run it on GPU nodes or scale it independently; the API warns when no layout worker is online.
Regex entities; no embeddings	No key-value extraction, handwriting or semantic enrichment; vectors are left to the client.	<code>chunks.jsonl</code> is the hand-off. A local embedding worker (e.g. bge-m3) could be added behind another queue.

Out of scope for a class project, and required before production: authentication with API keys or OAuth, per-tenant quotas and rate limiting, an allow-list for `callback_url` (SSRF risk), antivirus scanning (ClamAV) before processing, encryption at rest and retention policies (S3 lifecycle rules plus deleting DB rows), Alembic migrations instead of `create_all`, Prometheus metrics with alerting on queue age, and autoscaling workers on queue depth (e.g. KEDA).

11. Appendix · How to run & reproduce

```
# run (api :8000 with UI, /dashboard and /docs · flower :5555 · minio :9001)
docker compose up --build
docker compose --profile layout up -d --build          # optional Docling worker

# verify
docker compose run --rm --no-deps api pytest -q        # 47 unit tests
docker compose --profile test run --rm tests          # 40 integration tests (live stack)
python3 scripts/chaos_test.py                        # 13 failure injections
python3 scripts/load_test.py                         # burst load test

# benchmarks
docker build -t docintel-bench benchmark/
docker run --rm -v "$PWD/benchmark":/bench docintel-bench          # A
docker build -f benchmark/Dockerfile.structure -t docintel-bench-structure benchmark/
docker run --rm --add-host host.docker.internal:host-gateway \
-v "$PWD/benchmark":/bench docintel-bench-structure python run_structure_benchmark.py  # B
```

Path	Content
app/api	FastAPI app, schemas, dashboard aggregations
app/worker	Celery config, processing task, heartbeat, progress, retries, webhooks
app/jobs.py	dispatch-after-commit, atomic claim, recovery sweeper
app/extraction	format detection; PDF (+ layout → Markdown), image, text, Office, e-mail extractors; OCR engines; Docling pipeline; chunking; enrichment
benchmark/	dataset generator, runner, raw and summarised results, charts
samples/	demo documents (incl. structured PDF/DOCX, scanned tables, e-mail) and 11 edge cases
tests/, scripts/	unit and integration tests; chaos, load and sample-generation scripts
.github/workflows/ci.yml	CI on x86_64: build, unit, compose, integration, chaos, load
docs/	implementation plan, verification record, chaos and load results